

Reinforcement Learning Term Project: Two-Link Reacher (Control + RL)

Ali Majlesi — `ali.majlesi@sharif.edu`

Project Overview

In this term project you will design controllers and reinforcement learning agents for a **2-DOF planar reacher** (two-link robotic arm) with torque inputs (see Fig. 1). The goal is to move the end-effector to a target point (static or moving) in the plane.

In order to maintain a fair and computationally manageable workload, the required training duration for all algorithms is set to 100 episodes,

You will complete a sequence of parts:

- Build the environment (Gymnasium-style)
- Implement classical baselines (PID in task space; PID + inverse kinematics)
- Implement discrete-action RL (N-step SARSA)
- Implement continuous-action RL (DDPG) and run controlled experiments (exploration, generalization, ablation)
- Report standardized metrics and provide visualizations/videos

Submission Guidelines

1. **Packaging:** Zip your PDF file and all notebook files and name the archive as:

`RL_Project_studentNumber_studentName.zip`

If multiple notebooks exist, place them in a folder named **Notebooks** before zipping. Upload the zip file on **Quera**.

Final Grading Structure

The project will be evaluated according to the following grading breakdown:

Section	Weight
Environment Implementation	15%
Classical Controllers (PID, IK+PID)	15%
Discrete Reinforcement Learning (N-step SARSA)	15%
Continuous Reinforcement Learning (DDPG)	20%
Robustness & Generalization Experiments	20%
Analysis, Reporting, and Discussion	15%

Given Parameters

Use the following constants:

link parameters

$l_1 = 1.0$ m

$l_2 = 1.0$ m

$m_1 = 1.0$ Kg

$m_2 = 1.0$ Kg

damping

$b_1 = 0.05$ Kg.m/s

$b_2 = 0.05$ Kg.m/s

$dt = 0.02$ s

Gravity is ignored.

Notation:

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \quad \dot{q} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}, \quad \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}.$$

1 Part (a): Implement the Environment (Gymnasium)

1.1 Kinematics (End-Effector Position)

$$x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2), \quad y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2).$$

Define $p_{end} = (x, y)$ and target $p_{target} = (x_t, y_t)$.

1.2 Continuous-Time Dynamics (No Gravity)

The environment follows:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + B\dot{q} = \tau.$$

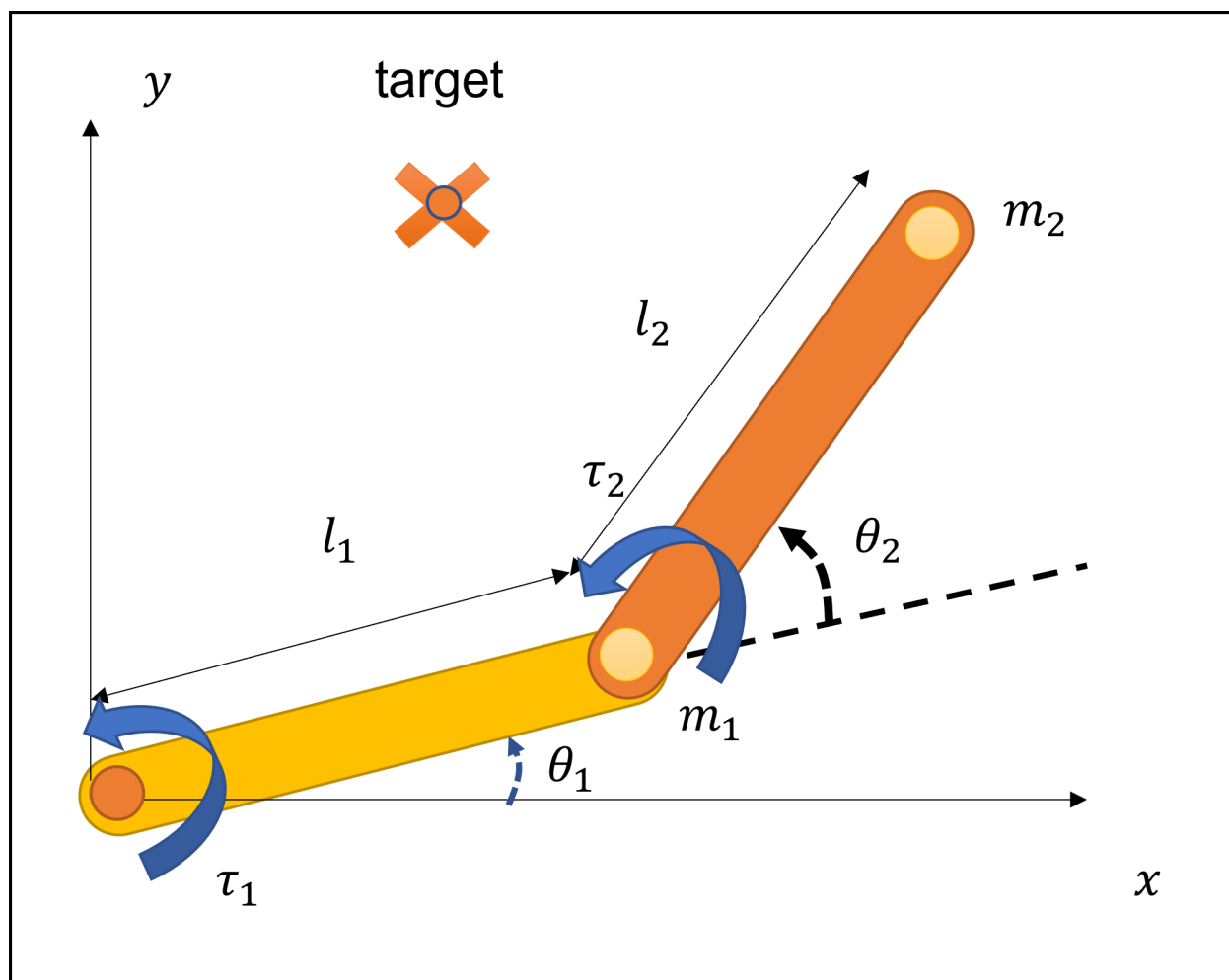


Figure 1: Reacher Diagram

Inertia Matrix

$$M(q) = \begin{bmatrix} a + 2b \cos \theta_2 & d + b \cos \theta_2 \\ d + b \cos \theta_2 & d \end{bmatrix}$$

with

$$a = m_1 l_1^2 + m_2 (l_1^2 + l_2^2), \quad b = m_2 l_1 l_2, \quad d = m_2 l_2^2.$$

Coriolis/Centrifugal Vector

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} -b \sin \theta_2 (2\dot{\theta}_1 \dot{\theta}_2 + \dot{\theta}_2^2) \\ b \sin \theta_2 \dot{\theta}_1^2 \end{bmatrix}.$$

Damping

$$B\dot{q} = \begin{bmatrix} b_1 \dot{\theta}_1 \\ b_2 \dot{\theta}_2 \end{bmatrix}.$$

Acceleration

$$\ddot{q} = M(q)^{-1} (\tau - C(q, \dot{q})\dot{q} - B\dot{q}),$$

and in code you must solve the 2×2 linear system (do not explicitly invert M).

1.3 Discretization with Timestep dt

Use semi-implicit Euler:

$$\dot{q}_{t+1} = \dot{q}_t + \ddot{q}_t dt, \quad q_{t+1} = q_t + \dot{q}_{t+1} dt.$$

(Optional) wrap angles to $[-\pi, \pi]$.

1.4 Loss and Reward

Loss:

$$\mathcal{L} = \|p_{\text{end}} - p_{\text{target}}\|^2 = (x - x_t)^2 + (y - y_t)^2.$$

Reward:

$$r_t = -\mathcal{L}.$$

1.5 Gymnasium API Requirements

Implement a Gymnasium-style environment class that follows the standard API:

- `__init__(dt=0.02, target=None, render_mode=None)` initializes the environment:
 - `dt`: simulation time step (default 0.02)
 - `target`: optional target position (if `None`, sample randomly)
 - `render_mode`: one of `{"human", "rgb_array"}`

- `reset` returns `(obs, info)`.
- `step(action)` returns `(obs, reward, terminated, truncated, info)`.
- `render()` behaves according to `render_mode`:
 - if `render_mode == "human"`: display the arm and target in a window and return `None`.
 - if `render_mode == "rgb_array"`: return an RGB image array of shape $(H, W, 3)$ (e.g., `uint8`).

Use a fixed horizon `max_steps = 150` (set `truncated=True` at the horizon).

1.6 Observation

Define and document the observation. A standard choice:

$$o = (x_t, y_t, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2). \quad (1)$$

Where (x_t, y_t) denotes the position of the target point.

Deliverables (Part a)

1. `reacher_env.py`: implementation of the environment.
2. Document dt , torque bounds, and `max_steps` in your report.
3. Test script: run one episode using actions $\tau_1 = 1 \text{ kg} \cdot \text{m}^2/\text{s}^2$ and $\tau_2 = 1 \text{ kg} \cdot \text{m}^2/\text{s}^2$, and render a video of the episode (you may use the `RecordVideo` wrapper from OpenAI Gymnasium).
4. `play_pygame.py`: a small interactive Pygame application to play the environment using the keyboard:
 - Create the environment with `render_mode="human"`.
 - Run the main loop at approximately $\frac{1}{dt}$ FPS.
 - Map keyboard keys to torques (example):
 - W/S: increase/decrease torque on joint 1
 - UP/DOWN: increase/decrease torque on joint 2
 - In each loop iteration, build the `action`, call `step(action)`, and then call `render()`.

2 Part (b): Cartesian-Space PID Control

In this part you will build a classical baseline controller using **PID in Cartesian space** using only:

$$p_{end}(t) = (x(t), y(t)), \quad p_{target}(t) = (x_t(t), y_t(t)).$$

No inverse kinematics is used.

2.1 Target Scenarios

- 1) **Static:** $p_{target} = (1.5, 0)$.
- 2) **Circular motion:** target rotates with

$$\omega = \frac{2\pi}{3} \cdot 0.2 \text{ rad/s}, \quad r = 0.5.$$

Choose a center (x_c, y_c) inside the workspace, and define

$$x_t(t) = x_c + r \cos(\omega t), \quad y_t(t) = y_c + r \sin(\omega t).$$

For this part, consider $(x_c, y_c) = (0, 0)$.

2.2 PID in Task Space

$$e(t) = p_{target}(t) - p_{end}(t),$$

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t).$$

Map to torques using Jacobian transpose:

$$\tau(t) = J(q(t))^T u(t).$$

The Jacobian is:

$$J(q) = \begin{bmatrix} -l_1 \sin \theta_1 - l_2 \sin(\theta_1 + \theta_2) & -l_2 \sin(\theta_1 + \theta_2) \\ l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2) & l_2 \cos(\theta_1 + \theta_2) \end{bmatrix}.$$

2.3 Evaluation

Compute average tracking error:

$$\bar{e} = \frac{1}{T} \sum_{t=1}^T \|p_{end}(t) - p_{target}(t)\|.$$

2.4 Video Requirement

Record a rollout video for both scenarios and embed both videos in your notebook.

Deliverables (Part b)

PID implementation, gains used, average tracking errors for both scenarios, and two embedded videos.

3 Part (c): Joint-Space PID + Inverse Kinematics

In this part you will solve inverse kinematics (IK) to compute desired joint angles (θ_1^d, θ_2^d) from target (x_t, y_t) , then use two PID controllers in joint space to track those desired angles.

3.1 Inverse Kinematics

Let $r^2 = x_t^2 + y_t^2$. Compute:

$$\cos \theta_2 = \frac{r^2 - l_1^2 - l_2^2}{2l_1l_2}, \quad \theta_2 = \arccos \left(\frac{x_t^2 + y_t^2 - l_1^2 - l_2^2}{2l_1l_2} \right).$$

Then

$$\alpha = \tan^{-1} \left(\frac{y_t}{x_t} \right), \quad \beta = \tan^{-1} \left(\frac{l_2 \sin \theta_2}{l_1 + l_2 \cos \theta_2} \right), \quad \theta_1 = \alpha - \beta.$$

(You may choose elbow-up or elbow-down.)

3.2 Joint-Space PID

$$\begin{aligned} e_1(t) &= \theta_1^d(t) - \theta_1(t), & \tau_1(t) &= K_{p1}e_1 + K_{i1} \int e_1 dt + K_{d1} \frac{de_1}{dt}, \\ e_2(t) &= \theta_2^d(t) - \theta_2(t), & \tau_2(t) &= K_{p2}e_2 + K_{i2} \int e_2 dt + K_{d2} \frac{de_2}{dt}. \end{aligned}$$

3.3 Targets, Evaluation, Videos

Use the same two target scenarios as Part (b), compute $\bar{e}$, and record/ embed two videos.

Deliverables (Part c)

IK implementation, PID implementation and gains, tracking errors for both scenarios, and embedded videos.

4 Part (d): Discrete Action Control with N -Step SARSA

4.1 Discrete Action Space

Discretize the torque vector:

$$\mathcal{A} = \{(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 0), (0, 1), (1, -1), (1, 0), (1, 1)\}.$$

Apply

$$\tau = \tau_{\max} a, \quad a \in \mathcal{A}.$$

4.2 N -Step SARSA

Implement on-policy N -step SARSA:

$$\begin{aligned} G_t^{(N)} &= \sum_{k=0}^{N-1} \gamma^k r_{t+k} + \gamma^N Q(s_{t+N}, a_{t+N}), \\ Q(s_t, a_t) &\leftarrow Q(s_t, a_t) + \alpha \left(G_t^{(N)} - Q(s_t, a_t) \right). \end{aligned}$$

4.3 Function Approximation

Because the state is continuous, use one:

- Tile coding (recommended),
- neural network approximation.

Deliverables (Part d)

Discrete-action wrapper, N -step SARSA implementation, learning curves, and final evaluation metrics.

5 Part (e): Continuous Control with DDPG

Deep Deterministic Policy Gradient (DDPG) is an off-policy, actor–critic algorithm designed for reinforcement learning problems with **continuous action spaces**. It combines ideas from Deterministic Policy Gradient (DPG) and Deep Q-Networks (DQN).

DDPG maintains two neural networks:

- **Actor network** $\mu(s|\theta^\mu)$: outputs a deterministic continuous action given a state.
- **Critic network** $Q(s, a|\theta^Q)$: estimates the Q-value of a state–action pair.

The critic is trained using the Bellman equation:

$$y = r + \gamma Q'(s', \mu'(s'|\theta^{\mu'})|\theta^{Q'})$$

where Q' and μ' are target networks used to stabilize training. The actor is updated using the policy gradient computed through the critic:

$$\nabla_{\theta^\mu} J \approx \mathbb{E} \left[\nabla_a Q(s, a|\theta^Q) \Big|_{a=\mu(s)} \nabla_{\theta^\mu} \mu(s|\theta^\mu) \right]$$

To improve stability, DDPG uses:

- Experience replay buffer
- Target networks with soft updates
- Exploration noise (e.g., Ornstein–Uhlenbeck noise)

DDPG is particularly useful in robotics and control tasks where actions are real-valued (e.g., torque control).

For a clear and intuitive explanation, see: **Video:** “Deep Deterministic Policy Gradient (DDPG) Explained” youtube or Aparat

5.1 State Input (Required)

For DDPG, the actor input must be:

$$s_t = (x_{target}, y_{target}, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2, x_{end}, y_{end}).$$

5.2 DDPG Requirements

Implement:

- Actor $\mu_\phi(s)$ and critic $Q_\psi(s, a)$
- Replay buffer
- Target networks and soft updates
- Mini-batch training

Deliverables (Part e)

Network architectures, training code, and learning curves (return/error vs steps).

6 Part (f): Exploration and Action Noise Study (DDPG-Specific)

DDPG is sensitive to exploration. Perform controlled experiments comparing:

1. Gaussian action noise:

$$a_t = \mu_\phi(s_t) + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2 I)$$

2. Ornstein–Uhlenbeck noise:

$$\epsilon_{t+1} = \epsilon_t + \theta(\mu - \epsilon_t)\Delta t + \sigma\sqrt{\Delta t}\eta_t, \quad \eta_t \sim \mathcal{N}(0, I)$$

Explanation of Noise Parameters

Below we explain the parameters appearing in each exploration equation in the context of DDPG.

1. Gaussian Action Noise

$$a_t = \mu_\phi(s_t) + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2 I)$$

Meaning of Terms

- a_t : Final action executed in the environment at time t . This is the deterministic policy output plus exploration noise.
- $\mu_\phi(s_t)$: Deterministic actor network parameterized by weights ϕ . It maps state s_t to an action and represents the pure policy output before noise is added.
- ϵ_t : Exploration noise added to the action. It is sampled independently at each time step to encourage exploration.

- $\mathcal{N}(0, \sigma^2 I)$: Multivariate Gaussian distribution with:
 - Mean 0, ensuring no directional bias.
 - Covariance matrix $\sigma^2 I$, where I is the identity matrix, meaning noise is independent across action dimensions.
 - σ^2 controls the variance (magnitude) of the noise.

Key Parameter

- σ (standard deviation): Controls exploration strength.
 - Large $\sigma \rightarrow$ stronger exploration.
 - Small $\sigma \rightarrow$ near-deterministic behavior.

It is common to decay σ over training.

Properties

- Noise is independent across time (no temporal correlation).
- Simple and stable.
- Widely used in modern actor-critic methods.

2. Ornstein–Uhlenbeck (OU) Noise

$$\epsilon_{t+1} = \epsilon_t + \theta(\mu - \epsilon_t)\Delta t + \sigma\sqrt{\Delta t}\eta_t, \quad \eta_t \sim \mathcal{N}(0, I)$$

This defines a temporally correlated stochastic process.

Meaning of Terms

- ϵ_t : Current noise value. Unlike Gaussian noise, it depends on the previous value, producing smooth and correlated exploration.
- ϵ_{t+1} : Next noise sample, computed using the previous value, a drift term, and a random perturbation.
- θ (mean reversion rate): Controls how strongly the noise returns to the mean μ .
 - Large $\theta \rightarrow$ fast reversion.
 - Small $\theta \rightarrow$ slower drift and more persistent noise.
- μ : Long-run mean of the process. Typically set to 0 in DDPG to center exploration.
- Δt : Time step size. Often set to 1 in discrete-time RL implementations.
- σ : Noise intensity (volatility), controlling randomness magnitude.
- $\eta_t \sim \mathcal{N}(0, I)$: Standard Gaussian random vector. Independent across time and injects stochasticity into the process.

Interpretation The OU process consists of three components:

$$\underbrace{\epsilon_t}_{\text{previous value}} + \underbrace{\theta(\mu - \epsilon_t)\Delta t}_{\text{mean-reverting drift}} + \underbrace{\sigma\sqrt{\Delta t}\eta_t}_{\text{random shock}}$$

Thus:

- The noise is temporally correlated.
- It evolves smoothly over time.
- It is often beneficial for physical control tasks with inertia (e.g., torque control).

Plot learning curves (mean $\pm$ std over $K = 3$ seeds) and success rate:

$$\text{SuccessRate} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}(\|p_{\text{end}}(t) - p_{\text{target}}(t)\| < \varepsilon).$$

Deliverables (Part f)

Plots for each exploration method.

7 Part (g): Generalization Tests (Target Distribution Shift)

Define a training target distribution (e.g., fixed circular motion or random static targets), then evaluate on shifted distributions:

- different ω ,
- different radius r and/or center (x_c, y_c) ,
- different trajectory shape (Fig. 2 / Lissajous).

$$x(t) = \sin\left(2t + \frac{\pi}{2}\right) \tag{2}$$

$$y(t) = \sin(4t) \tag{3}$$

Report average tracking error and success rate on each test distribution and compare to training performance.

Deliverables (Part g)

Train/test definitions, results table, and short generalization discussion.



Figure 2: Lissajous Curve.

8 Part (h): Ablation Study on State Inputs

Train three DDPG agents with different inputs:

1. Minimal:

$$s_t^{(1)} = (x_t, y_t, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$$

2. Minimal + end-effector:

$$s_t^{(2)} = (x_t, y_t, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2, x_{end}, y_{end})$$

3. Error-only:

$$s_t^{(3)} = (x_t - x_{end}, y_t - y_{end}, \theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2) \quad (4)$$

4. Error in Inverse-Kinematics:

$$s_t^{(3)} = (\theta_1 - \theta_{1_{target}}, \theta_2 - \theta_{2_{target}}, \dot{\theta}_1, \dot{\theta}_2) \quad (5)$$

Compare learning speed, final error across seeds.

Deliverables (Part h)

Learning curves, final table, and discussion of which representation works best.

9 Part (i): Metrics and Reporting Standards

For every method (PID, IK+PID, SARSA, DDPG), report:

1. Mean tracking error:

$$\bar{e} = \frac{1}{T} \sum_{t=1}^T \|p_{end}(t) - p_{target}(t)\|$$

2. Success rate:

$$\text{SuccessRate} = \frac{1}{T} \sum_{t=1}^T \mathbf{1}(\|p_{end}(t) - p_{target}(t)\| < \varepsilon)$$

3. Control energy:

$$E_\tau = \sum_{t=1}^T \|\tau_t\|^2$$

4. Smoothness:

$$S_{\Delta\tau} = \sum_{t=2}^T \|\tau_t - \tau_{t-1}\|^2$$

For RL methods, train with $K = 3$ random seeds and plot mean $\pm$ std curves.

Deliverables (Part i)

Standardized plots and a final comparison table across all methods.

Final Submission Checklist

Submit:

- Environment code (`reacher_env.py`) + tests
- Notebook including:
 - Part (b) and (c) videos embedded
 - SARSA and DDPG training plots
 - Exploration/generalization/ablation experiments
 - Final metrics table
- Short report (PDF) summarizing methods, results, and conclusions